

Chap 2

Introduction au MongoDB



mongoDB

M. Demba / Sidi Mahmoud RHIL

SupNum-Filière DSI-S4

MongoDB

- MongoDB (de l'anglais humongous = énorme)
 - Créé en 2008 par 10gen, première version publique en 2009
 - En 2013 10gen devient MongoDB Inc.
 - Base de données orientée document
 - Open-source
 - Développé en C++
 - Données stockées sous format JSON (BSON)
 - Absence de tables
 - Utilise la notion de « schemaless »
 - Fournit un shell javascript pour l'accès aux données et l'administration
 - Actuellement en version 8

MongoDB

- Avantages de MongoDB
 - Garantit la scalabilité horizontale (réplication et sharding)
 - Schéma flexible
 - La structure des documents est proche du modèle objet (réduit le défaut d'impédence)
 - Un langage de requêtage très riche
 - Supporte l'indexation pour optimiser les performances

Cas d'usage MongoDB

- A usage général: peut être utilisé dans une grande variété de cas d'usage et de secteurs
- Quelques exemples d'utilisation:
 - Applications web modernes (MEAN / MERN stacks)
 - Stockage et analyse de BigData (scalabilité horizontale, Aggregation Framework)
 - IoT: Stockage flexible pour des données issues de divers appareils avec des formats variables.

Les outils et commandes MongoDB

- Mongod: le moteur de base
- Mongosh: le shell javascript
- Mongos: le contrôleur de Sharding
- Les outils d'import/export
 - mongoimport, mongoexport, mongodump, mongorestore, bsondump
- mongofiles: pour interagir avec GridFS (système de fichier intégré à mongoDB)
- Mongostat: visualisation des stats d'une instance mongoDB
- mongotop: monitoring des temps d'accès par collection

Import des données

- JSON
 - `mongoimport -d <base> -c <collection> fichier.json`
- CSV
 - `Mongoimport --type csv --db <base> --collection <collection> --file fichier.csv --headerline`

Import des données

- JSON
 - `mongoimport -d <base> -c <collection> fichier.json --jsonArray`

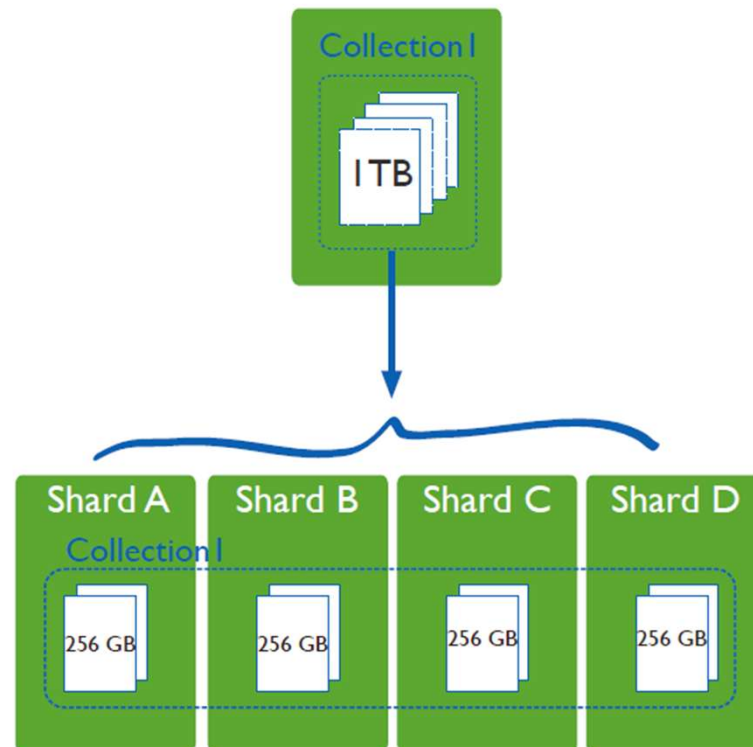
Ajouter l'option jsonArray si le fichier est un array : [...]

```
[
  { "_id": 8752, "title": "Divine Comedy", "author": "Dante", "copies": 1 },
  { "_id": 7000, "title": "The Odyssey", "author": "Homer", "copies": 10 },
  { "_id": 7020, "title": "Iliad", "author": "Homer", "copies": 10 },
  { "_id": 8645, "title": "Eclogues", "author": "Dante", "copies": 2 },
  { "_id": 8751, "title": "The Banquet", "author": "Dante", "copies": 2 }
]
```

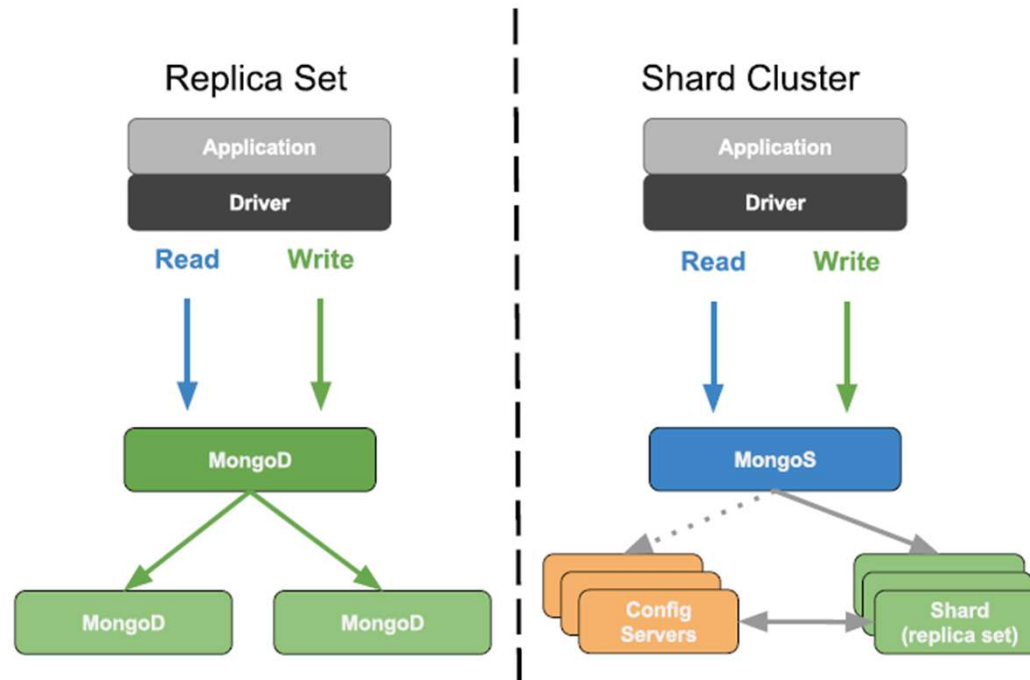
Clients graphiques

- MongoDB Compass :
 - Outil graphique officiel développé par MongoDB pour interagir avec des bases de données MongoDB (une version communautaire et une entreprise).
- Studio 3T:
 - Client puissant conçu pour faciliter le travail avec MongoDB
 - Une version gratuite et une payante avec plus de fonctionnalités
- Plugins pour les principaux IDE (vscode, intellij)

Scalabilité horizontale - sharding



Modèles de déploiement



MongoDB: un NoSQL orienté document



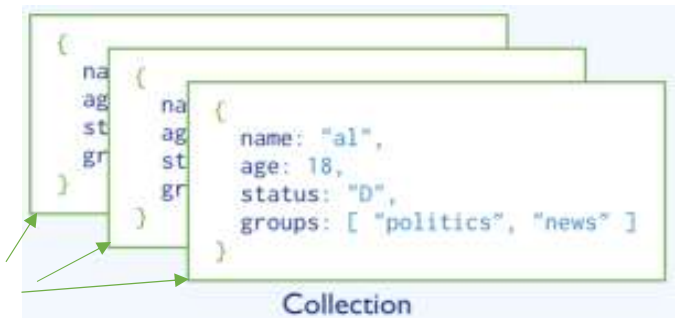
Orienté documents, Qu'est ce cela signifie ?

- ❖ Dans un SGBDR les données sont stockées par *ligne* dans des *tables*. Et il est souvent nécessaire de faire des jointures sur plusieurs tables afin de tirer des informations assez pertinentes de la base.
- ❖ Dans MongoDB, les données sont modélisées sous forme de *document* sous un style *JSON*.

```
{  
  "cle_1": valeur_1,  
  "cle_2": valeur_2,  
  ...  
  "cle_n": valeur_j  
}
```

"cle_i": clé/champ/attribut
Valeur_i: valeur

documents



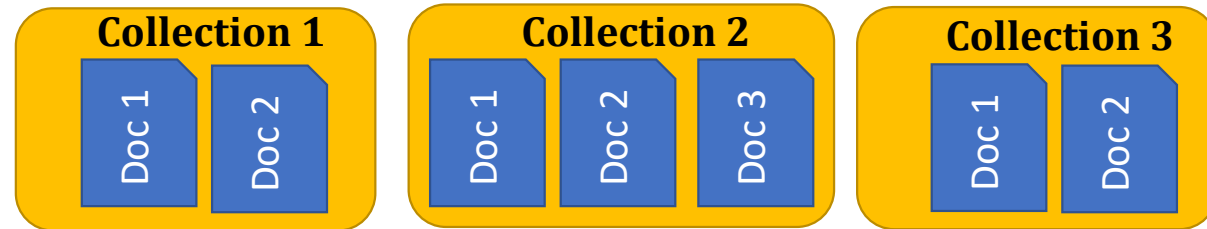
- ❖ On ne parle plus de tables, ni d'enregistrements mais de *collections* et de *documents*.
 - ❖ Ce système de gestion de données nous évite ainsi de faire des jointures de tables car toutes les informations propres à une certaine donnée sont stockées dans un même document.
-
- ✓ Les documents sont les unités de base dans une base MongoDB.
 - ✓ Ils sont équivalents aux objets JSON et sont comparables aux enregistrements d'une table dans une BDR.

Présentation de MongoDB

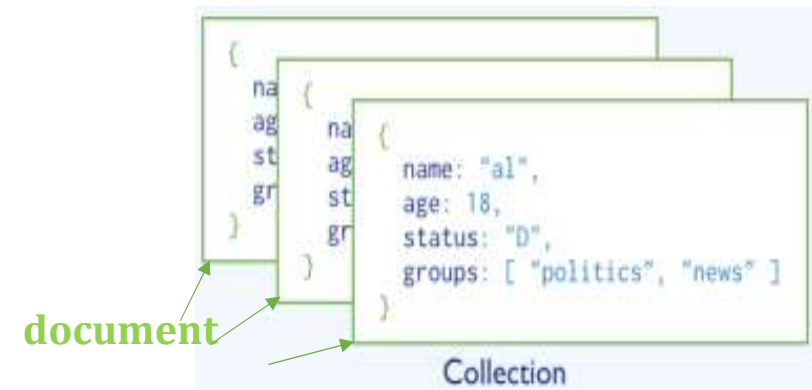
Structure d'un document en format JSON

```
{  
  "cle1": "abc",  
  "cle2": 123,  
  "cle3": false,  
  "cle4": [3,5,7,9],  
  "cle5": {  
    "cle51": "xyz",  
    "cle52": 67  
  }  
}
```

- Une collection est un ensemble de documents, l'équivalent d'une table en relationnel.
- Contrairement aux BDR, les champs des documents d'une collection sont libres et peuvent être différents d'un document à un autre.
- Le seul champ commun et obligatoire est le champ "_id".
- Pour que la base soit maintenable, il est préférable d'avoir dans une collection des documents de même type.



Les données sont stockées dans des documents
qui sont à leur tour stockés dans des collections



Concepts généraux

➤ **Base de données** (~ base de données en relationnel)

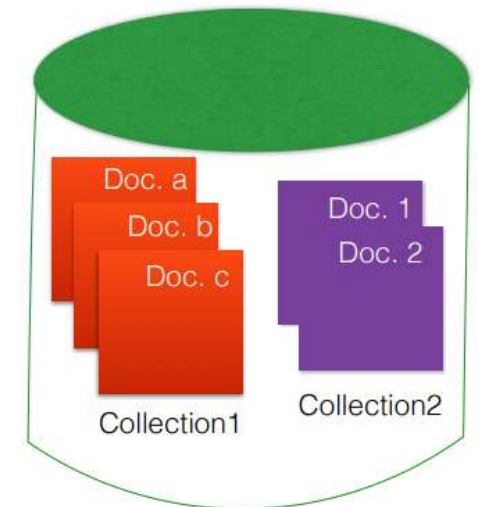
- ✓ Ensemble de collections
- ✓ Espace de stockage

➤ **Collection** (~ table en relationnel)

- ✓ Ensemble de documents qui partagent un objectif ou des similarités
- ✓ Pas de schéma prédéfini

➤ **Document** (~ ligne, tuple en relationnel)

- ✓ Un enregistrement (tuple) dans une collection



Base de données

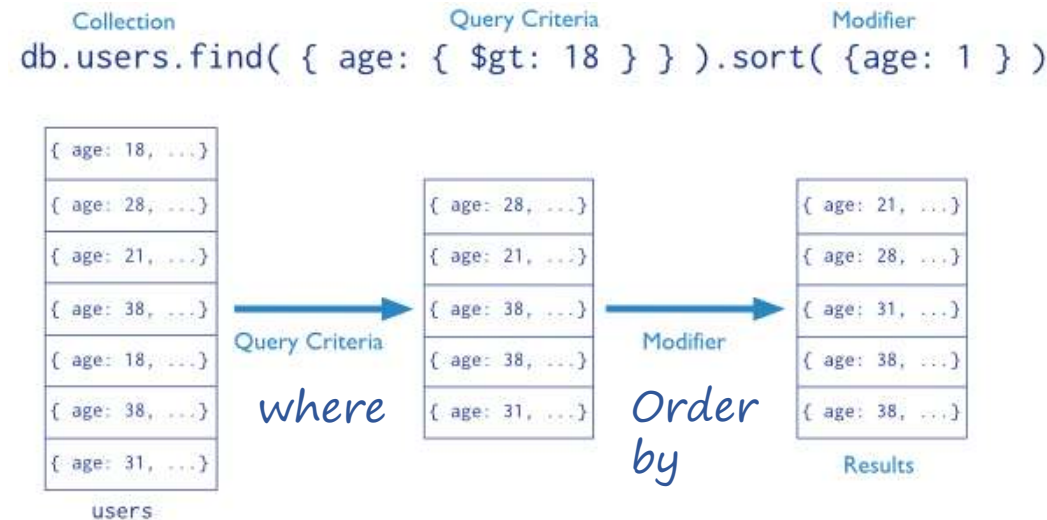
BDR		MongoDB
Base de donnée	➡	Base de données
Table, Vue	➡	Collection
Tuple	➡	Document (JSON, BSON)
Colonne	➡	Champs
Index	➡	Index
Jointure	➡	Document imbriqués
Clé étrangère	➡	Référence
Partition	➡	Shard

Les requêtes dans MongoDB

- ✓ Une requête porte sur une collection
- ✓ On pourra spécifier dans la requête des conditions et des critères qui permettent d'identifier le(s) document qui sera retourné au client.
- ✓ Un modifier est une méthode qui modifie la façon dont les résultats sont retournés, sans changer le filtre.

db.mycollection.**find**(arg1[, arg2])
pour faire une projection dans une collection (*select*)

- arg1 est un objet JSON qui permet de faire une restriction (*where*)
- arg2 : objet JSON contenant les champs à projeter



*SELECT age
FROM users
WHERE age>18
ORDER BY age ASC*

NoSQL

```
mydb> db.cmovies.find()
[
  {
    _id: 1,
    nom: 'Training Day',
    acteurs: [
      { nom: 'Washington', prenom: 'Denzel' },
      { nom: 'Hawke', prenom: 'Ethan' }
    ]
  },
  {
    _id: 2,
    nom: 'GoodFellas',
    acteurs: [
      { nom: 'Deniro', prenom: 'Robert' },
      { nom: 'Liota', prenom: 'Ray' }
    ]
  },
  {
    _id: 3,
    nom: 'Iron Man',
    acteurs: [ { nom: 'Downey', prenom: 'Robert' } ]
  }
]
```

SGBDR

Acteur			Film		Film_Acteur		
Id	Nom	Prenom	Id	Nom	Id	Id_fil m	Id_acteu r
1	Wahsington	Denzel	1	Training Day	1	1	1
2	Downey	Robert	2	GoodFellas	2	3	2
3	Hawke	Ethan			3	1	3
4	Deniro	Robert	3	Iron Man	4	2	4
5	Liota	Ray			5	2	5

Question: Quels sont les noms des acteurs du film GoodFellas ?

- ❖ Dans les SGBDR, on est obligé de faire une requête avec des jointures sur les trois tables
- ❖ Dans MongoDB, chaque document correspond à un film avec tous ses acteurs



plus nécessaire de faire de jointures afin de récupérer les informations sur un film

Question: Quels sont les noms des acteurs du film GoodFellas ?

❖ Dans les SGBDR

```
Select a.nom
From Acteur a, Film f, Film_Acteur fa
Where a.id=fa.id and
      f.id=fa.id and
      f.nom=<<GoodFellas>>;
```

Acteur			Film		Film_Acteur		
Id	Nom	Prenom	Id	Nom	Id	Id_film	Id_acteur
1	Washington	Denzel	1	Training Day	1	1	1
2	Downey	Robert	2	GoodFellas	2	3	2
3	Hawke	Ethan			3	1	3
4	Deniro	Robert	3	Iron Man	4	2	4
5	Liota	Ray			5	2	5

❖ Dans MongoDB

```
db.cmovies.find({'nom':'GoodFellas'},{'acteurs.nom':1}).pretty()
```

Résultat

```
> db.cmovies.find({nom:"GoodFellas"},{"acteurs.nom":1})
< {
  _id: 2,
  acteurs: [
    {
      nom: 'Deniro'
    },
    {
      nom: 'Liota'
    }
  ]
}
```

```
{
  "_id": ObjectID("yyyy..."),
  "nom": "GoodFellas",
  "acteurs": [
    {
      "nom": "Deniro",
      "prenom": "Robert"
    },
    {
      "nom": "Liota",
      "prenom": "Ray"
    }
  ]
}
```


Quelques commandes de base de MongoDB

créer une BD db1

```
test> use db1
```

créer deux collections col1 et col2

```
db1> db.createCollection("col1")
```

```
db1> db.createCollection("col2")
```

consulter les collections

```
db1> show collections
```

```
col1
```

```
col2
```

insérer 2 documents dans col1

```
col> db.col1.insertOne({name:"toto",prenom:"jojo"})
```

```
col> db.col1.insertOne({name:"titi",prenom:"jiji",age:25 })
```

consulter le nbre de documents dans col1

```
col> db.col1.countDocuments()
```

```
2
```

```
col> db.col1.find()
```

```
[
```

```
{
```

```
  _id: ObjectId("63e41e7dafaec47da81a1dff"),
```

```
  name: 'toto',
```

```
  prenom: 'jojo'
```

```
},
```

```
{
```

```
  _id: ObjectId("63e41ea5afaec47da81a1e00"),
```

```
  name: 'titi',
```

```
  prenom: 'jiji'
```

```
}
```

```
]
```

```
col>
```

Quelques commandes de base de MongoDB

afficher toutes les bases de données

`show dbs`

utiliser la base "madatabase" . Si la base n'existe pas, elle sera créée

`use madatabase`

voir dans quelle base on est

`db`

afficher les collections

`show collections`

compter le nombre de documents dans une collection

`db.col.countDocuments()`

renommer une collection

`db.oldname.renameCollection("newname")`

supprimer une collection

`db.contacts.drop()`

supprimer la base dans laquelle on est

`db.dropDatabase()`

insertion

`db.contacts.insertOne({ first: 'toto', last: 'tata' })`

sélection

`db.contacts.find()`

Avantages du MongoDB

- ✓ Plus proche des langages de programmation
- ✓ Très flexible grâce au modèle orienté documents
 - ✓ Possibilité d'avoir des documents de schémas différents dans une même collection ("Schemaless"), ce qui est pratique au début d'un projet par exemple car on est souvent amené à changer de schémas
- ✓ accès aux données plus rapides que le SQL par l'absence de jointures
- ✓ facile de gérer de très grandes quantités de données grâce au scaling (répartitions du moteur de BD sur plusieurs ordinateurs)
- ✓ la console mongo exécute du code JavaScript, ce qui est pratique pour insérer des données avec des fonctions JavaScript

Limites de MongoDB

- ☹ Le prix à payer pour la flexibilité est la non existence de clefs étrangères. Cela nous oblige à contrôler l'intégrité des données dans la couche applicative.
- ☹ À cause de la dénormalisation, on est exposé à la redondance.
- ☹ Si on reprend notre exemple sur les films, on serait obligé d'enregistrer un acteur dans chacun des films dans lesquels il jouerait.

MongoDB vs MySQL

	<u>MongoDB</u>	<u>MySQL</u>
SGBD	un puissant SGBD non relationnelle open source et gratuit, populaire pour le stockage de gros volumes de données.	un SGBDR gratuit et open source
Flexibilité des schémas	offre un schéma flexible qui permet aux utilisateurs de modifier la conception en fonction des exigences, notamment pour les applications Big Data	vous devez définir clairement les tables avant de stocker des données. Donc moins de flexibilité et déploiement plus lent
Format de stockage	stocke les données dans un format non structuré dans des collections → <i>donc format plus facile à modifier.</i>	basée sur les tables
Langage de requête utilisé	utilise le langage de requête MongoDB (MQL) au lieu de <u>SQL</u> . Il est expressif, riche et prend en charge les fonctions CRUD	utilise SQL
Performance et vitesse	Il stocke les données dans un seul document pour une entité et permet une lecture/écriture plus rapide des données → <i>MongoDB est relativement plus rapide et plus performant que MySQL</i>	si vous voulez modifier des données ou les extraire, vous devez passer par de nombreuses tables pour écrire/lire des données, ce qui augmente la charge sur le serveur et affecte ses performances.
Sécurité	Il exploite les contrôles d'accès basés sur les rôles avec des autorisations (read, write, etc.) flexibles pour les utilisateurs	Il dispose de contrôles d'accès basés sur les privilèges (select, delete, etc.).
Évolutivité	Il est extrêmement évolutif, ce qui est l'une des principales raisons pour lesquelles il est utilisé dans les sites web en pleine croissance, les boutiques eCommerce.	l'évolutivité est limitée
Facilité d'utilisation	Le processus de stockage des données de MongoDB est assez simple	MySQL est plus facile à utiliser en raison de sa simplicité et de sa structure adéquate.
Part de marché et popularité	Il est la BD non relationnelle la plus populaire et a reçu un bon accueil sur le marché global des BD. Selon le rapport de Datanyze, MongoDB se classe au 7 ^{ème} rang des BD avec une part de marché de 4,5 %. Il souligne également que plus de 7000 entreprises à travers le monde utilisent MongoDB.	MySQL est l'une des bases de données les plus populaires au monde. Selon le rapport de Statista, en juin 2021, MySQL est le 2 ^{ème} SGBD le plus populaire au monde après Oracle.

Modélisation: Exemple 1

One-to-One Linked



```
Book = {
  "_id": 1,
  "title": "Harry Potter and the Methods of Rationality",
  "slug": "9781857150193-hpmor",
  "author": 1,
  // more fields follow...
}

Author = {
  "_id": 1,
  "firstName": "Eliezer",
  "lastName": "Yudkowsky",
  "book": 1,
  // more fields follow...
}
```

Ou bien



```
Book = {
  "_id": 1,
  "title": "Harry Potter and the Methods of Rationality",
  "slug": "9781857150193-hpmor",
  "author": {
    "firstName": "Eliezer",
    "lastName": "Yudkowsky"
  },
  // more fields follow...
}
```

One-to-Many: Array in Parent



```
Author= {
  "_id": 1,
  "firstName": "Eliezer",
  "lastName": "Yudkowsky",
  "books": [1, 5, 17],
  // more fields follow...
}
```

et

One-to-Many: Scalar in Child

```
Book1= {
  "_id": 1,
  "title": "Harry Potter and the Methods of Rationality",
  "slug": "9781857150193-hpmor",
  "author": 1,
  // more fields follow...
}

Book2= {
  "_id": 5,
  "title": "How to Actually Change Your Mind",
  "slug": "1939311179490-how-to-change",
  "author": 1,
  // more fields follow...
}
```

Modélisation: Exemple 1

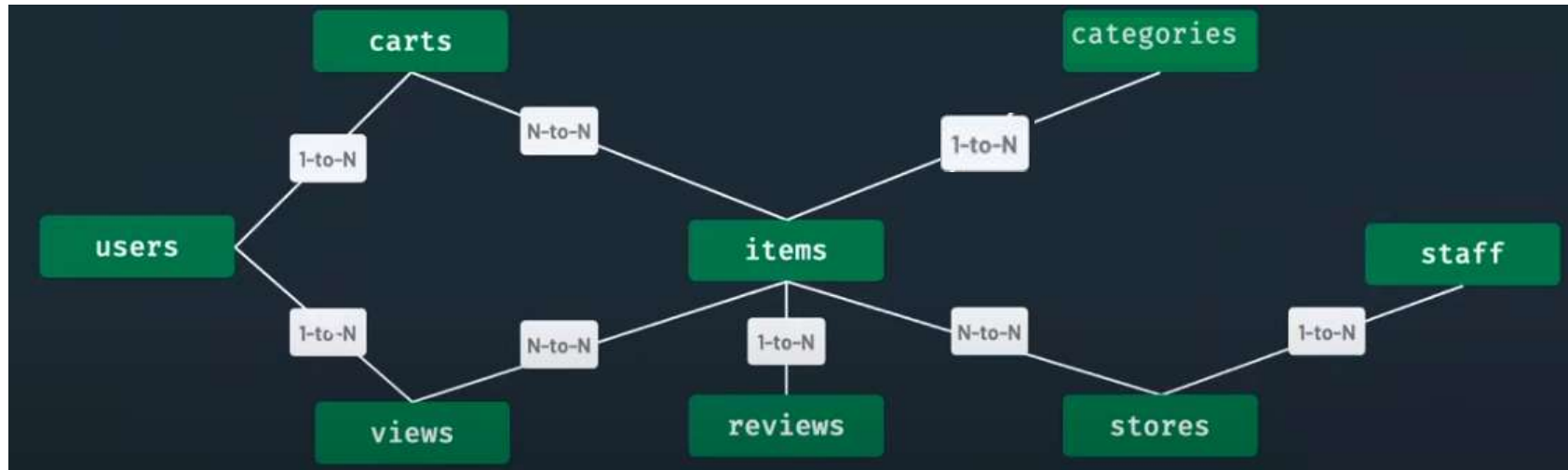
Many-to-Many: Arrays on either side

```
Book = {
  "_id": 5,
  "title": "Harry Potter and the Methods of Rationality",
  "slug": "9781857150193-hpmor",
  "authors": [1, 3],           // more fields follow...
}

Author = {
  "_id": 1,
  "firstName": "Eliezer",
  "lastName": "Yudkowsky",
  "books": [5, 7],            // more fields follow...
}
```

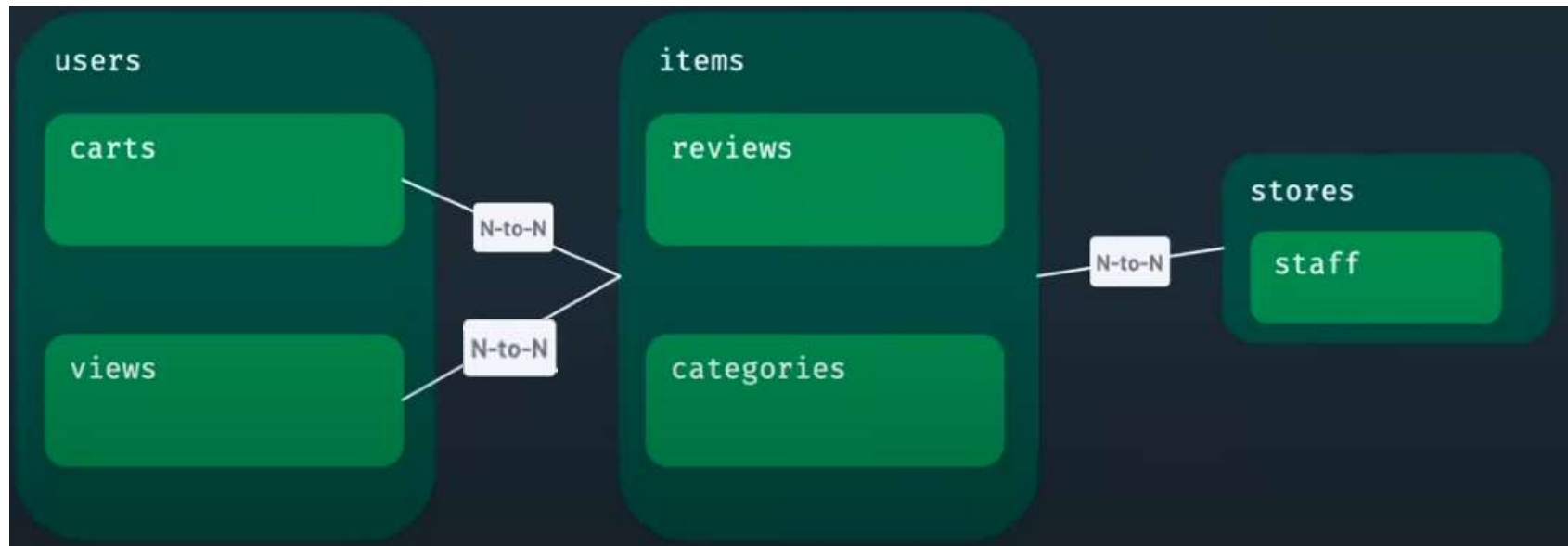


Collection Relationship Diagram or ERD



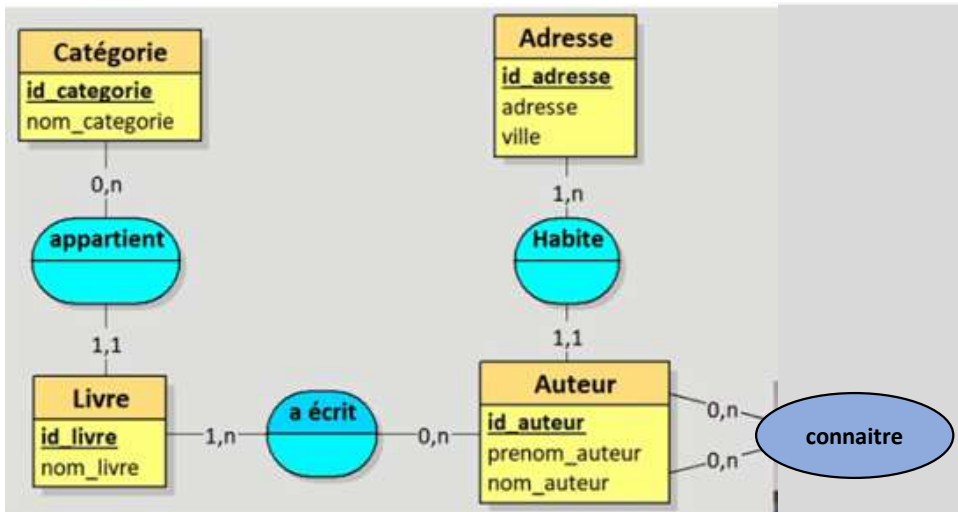
Modélisation:
Exemple 2

Collection Relationship Diagram or CRD (Embedded version: Better)

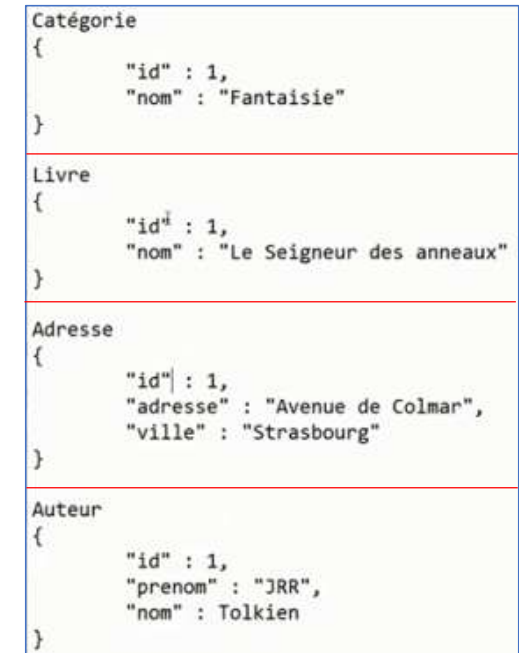


Modélisation: Exemple 3

ERD



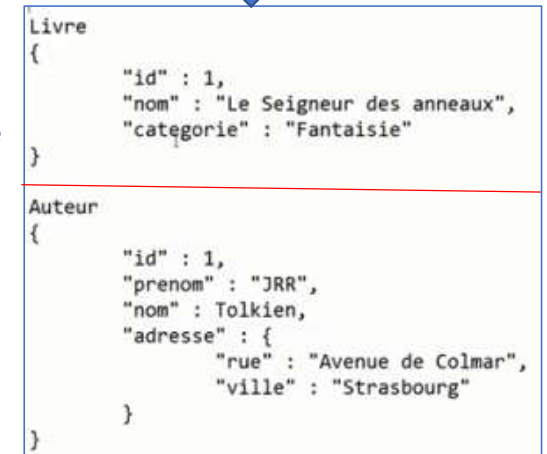
CRD

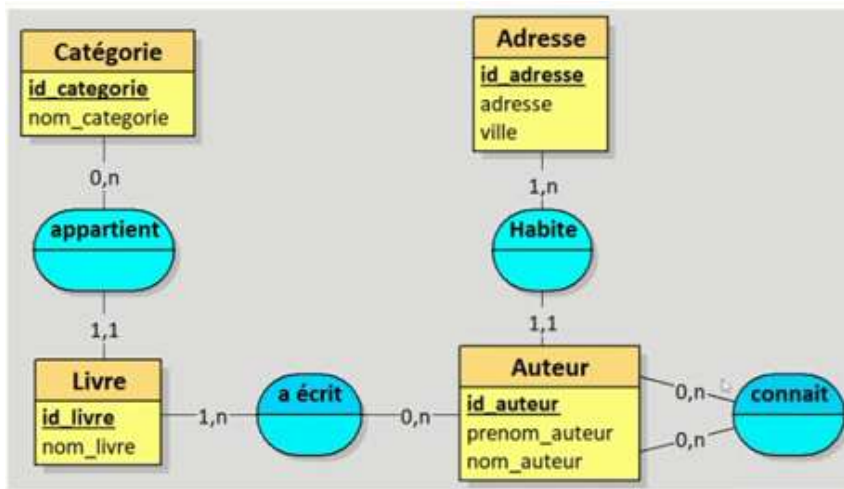


Transformer les entités en documents

Solution 1

En supposant qu'on n'aura pas besoin de chercher par catégorie ou adresse





Solution 1

Solution 2

```

Livres
{
  "id" : 1,
  "nom" : "Le Seigneur des anneaux",
  "categorie" : "Fantaisie"
}

Auteur
{
  "id" : 1,
  "prenom" : "JRR",
  "nom" : "Tolkien",
  "adresse" : {
    "rue" : "Avenue de Colmar",
    "ville" : "Strasbourg"
  }
}

```

On conserve Catégorie

```

Catégorie
{
  "id" : 1,
  "nom" : "Fantaisie"
}

Livres
{
  "id" : 1,
  "nom" : "Le Seigneur des anneaux",
  "categorie" : 1
}

Livres
{
  "id" : 1,
  "nom" : "Le Seigneur des anneaux",
  "categorie" : 1
}

Auteur
{
  "id" : 1,
  "prenom" : "JRR",
  "nom" : "Tolkien",
  "adresse" : {
    "rue" : "Avenue de Colmar",

```

Différence:

- ✓ Avec la solution 1, si on modifie la catégorie, il faut modifier dans tous les livres de cette catégorie alors que pour la solution 2, on modifie une seule fois.
- ✓ Si la catégorie est souvent modifiée, alors on opte la solution 2, sinon on opte la solution 1 qui prendra moins de temps pour modifier.

★ Pour la relation N-M entre Livre et Auteur.

L'auteur numéro 1 a écrit les livres numéros 1, 5 et 7,

Le livre numéro 1 est écrit par l'auteur numéro 1 et peut être d'autres

On pourrait également écrire les documents des livres
1, 5, 7 avec chacun ses auteurs et garder l'auteur tel quel => double references

```
Livre
{
  "id" : 1,
  "nom" : "Le Seigneur des anneaux",
  "categorie" : "Fantaisie",
  "auteur" : [1]
}

Auteur
{
  "id" : 1,
  "prenom" : "JRR",
  "nom" : "Tolkien",
  "adresse" : {
    "rue" : "Avenue de Colmar",
    "ville" : "Strasbourg"
  },
  "livre" : [1, 5, 7]
}
```

★ Pour la relation réflexive Connaissance entre les Auteurs.

Si l'auteur 1 connaît l'auteur 2, alors l'auteur 2 connaît l'auteur 1.

```
Auteur
{
  "id" : 1,
  "prenom" : "JRR",
  "nom" : "Tolkien",
  "adresse" : {
    "rue" : "Avenue de Colmar",
    "ville" : "Strasbourg"
  },
  "livre" : [1, 5, 7],
  "connaissance" : [2, 7, 11]
}

Auteur
{
  "id" : 2,
  "...",
  "connaissance" : [1, 7, 11]
}
```

Recap

- **1- n**

Si n est petit(few), alors on opte embedding sinon referencing

- **n- m**

On preferera le referencing

MongoDB Schema Design Rules

- Favor embedding unless there is a compelling reason not to
- Needing to access an object on its own is a compelling reason not to embed it
- Avoid JOINS and \$lookups if they can be avoided
- Arrays should not grow without bound
- How you model your data depends - entirely - on your particular application's data access patterns

JOE
Karlsson

- Relational vs. MongoDB Schema Design Approaches
 - Relational Schema Design
 - Model data independent of queries
 - Normalize in the 3rd form
 - MongoDB Schema Design 
 - No rules, no process, no algorithm
 - Considerations:
 - How to store the data
 - Query Performance
 - Design a schema that works for your application

Les opérateurs

Les opérateurs **\$eq**, **\$ne**, **\$lt**, **\$gt**, **\$lte**, **\$gte** s'utilisent de la même façon en MongoDB. Ces opérateurs comparent la valeur d'une variable à une valeur fixe (nombre, booléen, chaîne de caractères...), comme dans l'exemple suivant :

Opérateur logique	Mot clé en MongoDB
=	\$eq
≠	\$ne
<	\$lt
>	\$gt
≤	\$lte
≥	\$gte
∈	\$in
∉	\$nin
clé existante	\$exists
.	\$size

SQL

```
SELECT * FROM notes  
WHERE notes >= 13
```

```
SELECT * FROM T  
WHERE a IN ("chaîne1", "chaîne2")
```

MongoDB

```
db.notes.find({"notes": {$gte: 13}})
```

```
db.T.find( { "a": { $in: ["chaîne1", "chaîne2"] } } )
```

Les opérateurs

Afficher les documents validant les deux conditions suivantes : a=1 et b=5.

```
SELECT * FROM T  
WHERE a = 1 and b = 5
```

```
db.T.find( { "a": 1, "b": 5 } )
```

```
SELECT *  
FROM t  
WHERE a=1 or b=5
```

```
$or : [ {condition 1}, ... , {condition n} ].
```

```
db.t.find(  
    { $or : [ { "a": 1 }, { "b": 5 } ] }  
)
```

```
SELECT *  
FROM t  
WHERE a<>1 or b <> "bleu "
```

```
db.collectionName.find(  
    { $nor : [ { "a": 1 }, { "b": "blue" } ] }  
)
```

Les requêtes

Afficher les noms des étudiants sans le *_id*:

```
db.notes.find({}, {"nom": true, "_id": false})
```

Afficher la liste des étudiants ayant obtenu exactement deux notes ?

```
db.notes.find( {"nom": { $size: 2 } } )
```

retourne tous les documents où le champ *nom* est un tableau **avec deux éléments**.

Par exemple la requête retourne

{nom : [10.0, 12.5] } et {nom : [0.0, 8.75] }

mais pas {nom : [05.0] } ou {nom : [12.0, 8.75, 14.5] }

Les requêtes de modification: insertXxx(), updateXxx()

On se donne la collection

people: ({ **user_id**: "abc123", **age**: 55, **status**: 'A' })

MySQL

```
CREATE TABLE people (  
  id MEDIUMINT NOT NULL  
    AUTO_INCREMENT,  
  user_id VARCHAR(30),  
  age Number,  
  status CHAR(1),  
  PRIMARY KEY (id)  
)
```

```
ALTER TABLE people  
ADD join_date DATETIME
```

```
ALTER TABLE people  
DROP COLUMN join_date
```

+ un *insert into*

MongoDB

```
db.people.insertOne( {  
  user_id: "abc123",  
  age: 55,  
  status: "A"  
} )
```

```
db.people.updateMany(  
  { },  
  { $set: { join_date: new Date() } }  
)
```

```
db.people.updateMany(  
  { },  
  { $unset: { "join_date": "" } }  
)
```

On peut aussi créer explicitement une collection

```
db.createCollection("people")
```

Insertion

MySQL

```
INSERT INTO people(user_id, age, status)
VALUES ("bcd001", 45, "A")
```

MongoDB

```
db.people.insertOne(
  { user_id: "bcd001", age: 45, status: "A" }
)
```

Sélection

MySQL

```
SELECT *
FROM people
```

```
SELECT id,
       user_id,
       status
FROM people
```

```
SELECT user_id, status
FROM people
```

MongoDB

```
db.people.find()
```

```
db.people.find(
  { },
  { user_id: 1, status: 1 }
)
```

```
db.people.find(
  { },
  { user_id: 1, status: 1, _id: 0 }
)
```


Sélection

MySQL

```
SELECT *  
FROM people  
WHERE status = "A"
```

```
SELECT user_id, status  
FROM people  
WHERE status = "A"
```

```
SELECT *  
FROM people  
WHERE status != "A"
```

```
SELECT *  
FROM people  
WHERE status = "A"  
AND age = 50
```

MongoDB

```
db.people.find(  
  { status: "A" }  
)
```

```
db.people.find(  
  { status: "A" },  
  { user_id: 1, status: 1, _id: 0 }  
)
```

```
db.people.find(  
  { status: { $ne: "A" } }  
)
```

```
db.people.find(  
  { status: "A",  
    age: 50 }  
)
```

Sélection

MySQL

```
SELECT *  
FROM people  
WHERE age > 25
```

```
SELECT *  
FROM people  
WHERE age < 25
```

```
SELECT *  
FROM people  
WHERE age > 25  
AND age <= 50
```

```
SELECT *  
FROM people  
WHERE user_id like "%bc%"
```

```
SELECT *  
FROM people  
WHERE user_id like "bc%"
```

MongoDB

```
db.people.find(  
  { age: { $gt: 25 } }  
)
```

```
db.people.find(  
  { age: { $lt: 25 } }  
)
```

```
db.people.find(  
  { age: { $gt: 25, $lte: 50 } }  
)
```

```
db.people.find( { user_id: /bc/ } )
```

-or-

```
db.people.find( { user_id: { $regex: /bc/ } } )
```

```
db.people.find( { user_id: /^bc/ } )
```

-or-

```
db.people.find( { user_id: { $regex: /^bc/ } } )
```

Select *
From people
Where user_id like '%ro'

```
db.people.find({'user_id': /ro$/})
```

Sélection

MySQL

```
SELECT *  
FROM people  
WHERE status = "A"  
ORDER BY user_id ASC
```

```
SELECT *  
FROM people  
WHERE status = "A"  
ORDER BY user_id DESC
```

```
SELECT COUNT(*)  
FROM people
```

```
SELECT COUNT(user_id)  
FROM people
```

MongoDB

```
db.people.find( { status: "A" } ).sort( { user_id: 1 } )
```

```
db.people.find( { status: "A" } ).sort( { user_id: -1 } )
```

```
db.people.count()
```

or

```
db.people.find().count()
```

```
db.people.count( { user_id: { $exists: true } } )
```

Sélection

MySQL

```
SELECT COUNT(*)  
FROM people  
WHERE age > 30
```

```
SELECT DISTINCT(status)  
FROM people
```

```
SELECT *  
FROM people  
LIMIT 1
```

MongoDB

```
db.people.count( { age: { $gt: 30 } } )
```

or

```
db.people.find( { age: { $gt: 30 } } ).count()
```

```
db.people.aggregate( [ { $group : { _id : "$status" } } ] )
```

or

```
db.people.distinct( "status" )
```

```
db.people.findOne()
```

or

```
db.people.find().limit(1)
```

Sélection

MySQL

```
SELECT *  
FROM people  
LIMIT 5  
SKIP 10
```

```
EXPLAIN SELECT *  
FROM people  
WHERE status = "A"
```

MongoDB

```
db.people.find().limit(5).skip(10)
```

```
db.people.find( { status: "A" } ).explain()
```

Mise à jour

MySQL

```
UPDATE people  
SET status = "C"  
WHERE age > 25
```

```
UPDATE people  
SET age = age + 3  
WHERE status = "A"
```

MongoDB

```
db.people.updateMany(  
  { age: { $gt: 25 } },  
  { $set: { status: "C" } }  
)
```

```
db.people.updateMany(  
  { status: "A" },  
  { $inc: { age: 3 } }  
)
```

Suppression

MySQL

```
DELETE FROM people  
WHERE status = "D"
```

```
DELETE FROM people
```

MongoDB

```
db.people.deleteMany( { status: "D" } )
```

```
db.people.deleteMany({})
```

TD01: Installation MongoDB

Exercice 1

1. Importez les fichiers notes.json et etudiant.json dans les collections note et etudiant de la BD **ecole**
c:\data\db>mongoimport --db ecole --collection note --file notes.json --jsonArray
c:\data\db>mongoimport --db ecole --collection etudiant --file etudiant.json --jsonArray
2. Affichez les documents de la collection note:
ecole> db.note.find()
[
 { _id: 1, idEtudiant: 1, note: 10, idMatiere: 1 },
 { _id: 2, idEtudiant: 2, note: 12, idMatiere: 2 },
 { _id: 5, idEtudiant: 4, note: 9, idMatiere: 1 },
 { _id: 6, idEtudiant: 1, note: 8, idMatiere: 3 },
 { _id: 8, idEtudiant: 3, note: 15, idMatiere: 1 },
 { _id: 9, idEtudiant: 1, note: 16, idMatiere: 2 },
 { _id: 7, idEtudiant: 2, note: 14, idMatiere: 3 },
 { _id: 10, idEtudiant: 2, note: 12, idMatiere: 1 },
 { _id: 4, idEtudiant: 4, note: 11.5, idMatiere: 2 },
 { _id: 3, idEtudiant: 3, note: 10.5, idMatiere: 3 }
]
3. Affichez les docs dont les notes sont supérieures ou égal à 10.
[
 { _id: 1, idEtudiant: 1, note: 10, idMatiere: 1 },
 { _id: 2, idEtudiant: 2, note: 12, idMatiere: 2 },
 { _id: 8, idEtudiant: 3, note: 15, idMatiere: 1 },
 { _id: 9, idEtudiant: 1, note: 16, idMatiere: 2 },
 { _id: 7, idEtudiant: 2, note: 14, idMatiere: 3 },
 { _id: 10, idEtudiant: 2, note: 12, idMatiere: 1 },
 { _id: 4, idEtudiant: 4, note: 11.5, idMatiere: 2 },
 { _id: 3, idEtudiant: 3, note: 10.5, idMatiere: 3 }
]

4. Affichez les docs dont les notes sont 10, 14, 15 ou 16.
[
 { _id: 1, idEtudiant: 1, note: 10, idMatiere: 1 },
 { _id: 8, idEtudiant: 3, note: 15, idMatiere: 1 },
 { _id: 9, idEtudiant: 1, note: 16, idMatiere: 2 },
 { _id: 7, idEtudiant: 2, note: 14, idMatiere: 3 }
]
5. Affichez la note de l'étudiant #2 obtenue dans la matière #1
[
 { _id: 10, idEtudiant: 2, note: 12, idMatiere: 1 }
]
6. Affichez seulement la note de l'étudiant #2 obtenue dans la matière #1
[
 { _id: 1, note: 12 }
]
7. Affichez les notes de la matière #1
[
 { _id: 1, note: 10 },
 { _id: 5, note: 9 },
 { _id: 8, note: 15 },
 { _id: 10, note: 12 }
]
8. Modifiez la requête précédente pour exclure le _id.
[{ note: 10 }, { note: 9 }, { note: 15 }, { note: 12 }]

9. Affichez les notes qui sont inferieures à 10, afficher la note et l'id matière.

```
[
  { _id: 5, note: 9, idMatiere: 1 },
  { _id: 6, note: 8, idMatiere: 3 }
]
```

10. Affichez les notes des matières #1 et #2.

```
[
  { _id: 1, note: 10 },
  { _id: 2, note: 12 },
  { _id: 5, note: 9 },
  { _id: 8, note: 15 },
  { _id: 9, note: 16 },
  { _id: 10, note: 12 },
  { _id: 4, note: 11.5 }
]
```

11. afficher la liste des étudiants dont le prénom commence par A et contient exactement 5 lettres

```
[
  {
    _id: 2,
    nom: 'Aicha',
    prenom: 'Ahmed',
    email: 'fAhmed1@netscape.com',
    gendre: 'Female',
    date_naiss: '5/19/2022'
  },
  {
    _id: 10,
    nom: 'Aysetou',
    prenom: 'Ahmed',
    email: 'cahmed9@businessins_ider.com',
    gendre: 'Female',
    date_naiss: '2/17/2022'
  }
]
```

12. Afficher le nom et la date de naissance de l'étudiant le plus âgé.



13. Attribuer la note 13,5 à l'étudiant #1 dans la matière #1.



14. Augmenter de +1 tous les étudiants qui ont des notes inférieures à 10.



15. Afficher les notes triées par ordre décroissant



16. Afficher les 3 meilleurs notes



17. Afficher les notes entre 10 et 15



18. Affichez les notes qui sont inférieures à 10 (note et idMatiere)



19. Ajouter le nouvel étudiant ci-dessous dans la collection etudiant

```
{ _id: 11, nom: "Sow", prenom: "Amadou", email: "amadou.sow@gmail.com", genre: "Male", date_naiss: "3/12/2022"}
```

20. Ajouter plusieurs notes pour l'étudiant #11 qui vient d'être ajouté avec une seule instruction

```
idMatiere: 1, note: 14
```

```
idMatiere: 2, note: 12
```

```
idMatiere: 3, note: 15
```

21. Supprimer la note de l'étudiant #4 dans la matière #1

22. Supprimer toutes les notes inférieures ou égales à 10

23. Supprimer l'étudiant #11